

Kalai — Food & Nutrition Tracking App

Project Specification

Course: Vývoj aplikací pro mobilní platformy **Authors:** Jan Herold (učo 550508), Jiří Krokviak (učo 582916) **Date:** March 2026

1. Abstract

Kalai is a cross-platform food and nutrition tracking application built with Kotlin Multiplatform (KMP) targeting Android and iOS. The app enables users to log their daily food intake through multiple input methods: AI-powered photo analysis, barcode scanning, text search, and manual entry.

The application leverages OpenAI's GPT vision model to analyze food photographs and automatically extract nutritional information (calories, protein, carbohydrates, fat). For packaged products, Kalai supports barcode scanning via Google ML Kit with product lookup from both a local database and the Open Food Facts API.

Users set personalized nutritional goals during an 11-step onboarding flow based on their body metrics (weight, height, age, gender, activity level) and fitness objectives (weight loss, maintenance, or gain). The app tracks daily progress toward calorie and macronutrient targets with visual progress indicators, maintains a streak counter for consecutive days of logging, and provides analytics with charts showing weight trends and nutrition data over configurable date ranges.

The architecture follows MVVM with all business logic (ViewModels, repositories, network clients, database) shared via KMP's `commonMain` module, while platform-specific UI (Jetpack Compose on Android, SwiftUI on iOS) and hardware access (camera, barcode scanner) remain in their respective platform modules.

2. Functional Requirements

2.1 Onboarding

An 11-step guided setup flow collects user preferences and body metrics to calculate personalized nutritional targets:

Step	Purpose	Input Method
1	Language selection	Segmented control (Čeština / English)
2	Unit system	Segmented control (Metric / Imperial)
3	Theme preference	Segmented control (System / Light / Dark)
4	Gender	Toggle (Male / Female)
5	Body weight	Wheel picker (unit-aware)
6	Body height	Wheel picker (unit-aware)
7	Age	Wheel picker (1–120 years)
8	Activity level	Selection (Sedentary / Light / Active / Very Active)
9	Fitness goal	Selection (Weight loss / Maintenance / Weight gain)
10	Macro targets	Auto-calculated from goal, manually adjustable
11	Promo code	Optional text input

The flow supports swipe navigation (left/right), displays a progress bar, and persists all data on completion.

2.2 Food Logging

Users can add food items through four methods:

- **Photo analysis (AI):** Capture a photo via CameraX → send raw image bytes to the backend → OpenAI GPT vision model analyzes the food and returns nutritional data (calories, protein, fat, carbs, health score 1–10) in Czech. A placeholder entry with a loading indicator is shown during analysis.
- **Barcode scanning:** ML Kit detects EAN-13, EAN-8, UPC-A, UPC-E barcodes → lookup in local backend SQLite database → fallback to Open Food Facts API. Users select portion size before adding.
- **Text search:** Search the local food database and Open Food Facts API. Results show food name, calories, and macros with portion size selection.
- **Manual entry:** Create custom foods from scratch or by combining existing foods as ingredients. Ingredient portions are adjustable with automatic macro recalculation.

2.3 Daily Tracking & Progress

- Calorie progress card showing current vs. target calories with a progress bar
- Three macronutrient cards (protein, carbs, fat) with circular progress indicators
- Week-based date picker for navigating between days
- List of recently added food items for the selected date
- Streak counter showing consecutive days with logged food

- Selection mode for bulk operations (save as custom food, delete)

2.4 Custom Food Management

- Create custom foods with name, calories, and macronutrient values
- Build composite foods from multiple ingredients with adjustable portions
- Browse and search user-created custom foods
- View recently used foods for quick re-logging
- Save selected food combinations as new custom foods

2.5 Food Detail Editing

- View and edit food name, protein, fat, and carbohydrate values
- View food photograph (if captured via camera)
- Re-analyze food image ("Fix Result") to get updated nutritional data
- Share or delete food entries via context menu

2.6 Nutrient Target Management

- View current calorie progress with visual indicator
- Adjust daily targets for protein, carbs, and fat (0–500g range)
- iOS-style inline wheel pickers for value selection
- Targets are auto-calculated during onboarding and manually adjustable afterward

2.7 Analytics

- Configurable date range selection with inline date pickers
- Weight trend line chart over selected date range
- Daily nutrition bar charts showing calorie and macronutrient totals
- Data sourced from historical personal info records and food item aggregates

2.8 Profile Management

- Edit weight, height, and age with inline pickers
- Gender selection (Male / Female)
- BMI calculation and category indicator
- Activity level selection (Sedentary / Light / Active / Very Active)

2.9 Settings

- **Theme:** System / Light / Dark appearance modes
- **Language:** Czech (Čeština) and English
- **Units:** Metric (kg, cm) and Imperial (lbs, in)
- **Notifications:** Toggle meal reminder notifications (local, time-gated 7:00–21:00 with 90-minute inactivity threshold)

- **App version** display

2.10 Meal Reminders

- Local notification system using WorkManager
- Time gate: active only between 7:00 and 21:00
- Inactivity gate: suppressed if food was logged within the last 90 minutes
- Reminder types:
- No food logged after 10:00 → remind to log food
- Behind on macro targets (< 60% of expected linear progress) → remind with progress percentage

2.11 Streak System

- Tracks consecutive days with at least one logged food item
- Displayed on the home screen calorie card
- Resets when a day is missed

3. Platform-Specific Functionality

3.1 Android

- **CameraX** (v1.4.1): Full camera preview with photo capture for food analysis
- **ML Kit Barcode Scanning** (v17.3.0): Real-time barcode detection supporting EAN-13, EAN-8, UPC-A, UPC-E formats
- **Push Notifications**: Local meal reminders via WorkManager, with runtime permission request on Android 13+ (POST_NOTIFICATIONS)
- **Image Storage**: File-based local image persistence via AndroidImageStorage

3.2 iOS (Skeleton)

- Platform module with DriverFactory (SQLDelight native driver) and KoinHelper for DI initialization
 - SwiftUI entry point prepared for future UI implementation
-

4. Technologies

4.1 Core Framework

Technology	Version	Purpose
Kotlin Multiplatform	2.1.10	Cross-platform shared module
Jetpack Compose	BOM 2025.02.00	Android declarative UI
Compose Multiplatform	1.7.3	Shared Compose resources
Material3	1.3.1	Design system and theming
Material Icons Extended	BOM	Icon library

4.2 Architecture & DI

Technology	Version	Purpose
Koin	3.2.0	Multiplatform dependency injection
AndroidX Lifecycle ViewModel	2.8.2	ViewModel lifecycle management
AndroidX Activity Compose	1.10.1	Compose activity integration

4.3 Data & Networking

Technology	Version	Purpose
SQLDelight	2.0.2	Multiplatform SQLite database
Ktor Client	3.1.0	HTTP client (OkHttp engine on Android, Darwin on iOS)
KotlinX Serialization	1.8.0	JSON serialization/deserialization
KotlinX DateTime	0.6.2	Multiplatform date/time handling
KotlinX Coroutines	1.10.1	Asynchronous programming

4.4 UI Libraries

Technology	Version	Purpose
Coil3	3.1.0	Async image loading
Vico	2.1.0	Multiplatform chart library
Haze	1.5.0	Blur/glassmorphism effects

Technology	Version	Purpose
Compose Navigation	2.8.9	Type-safe navigation with @Serializable routes
Compose Foundation	1.7.8	Snap fling behavior for wheel pickers

4.5 Platform Libraries

Technology	Version	Purpose
CameraX	1.4.1	Camera preview and photo capture
ML Kit Barcode	17.3.0	Barcode detection
WorkManager	2.10.0	Background task scheduling

4.6 Backend

Technology	Version	Purpose
Bun	latest	JavaScript/TypeScript runtime with built-in SQLite
OpenAI SDK	^4.73.0	GPT vision model API client
SQLite (FTS5)	built-in	Product database with full-text search

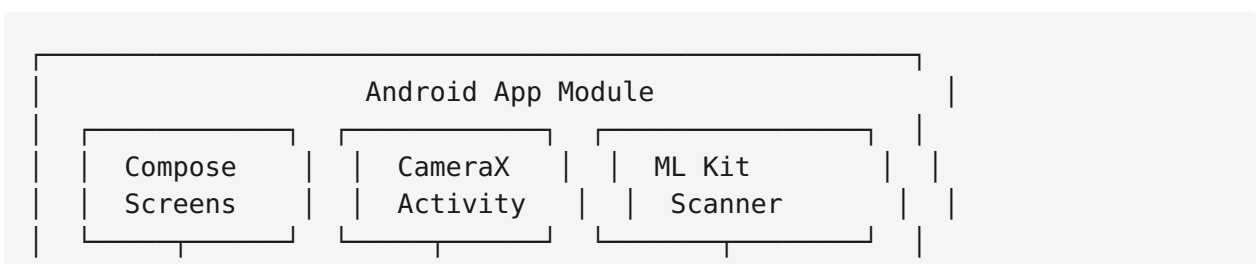
4.7 Build Requirements

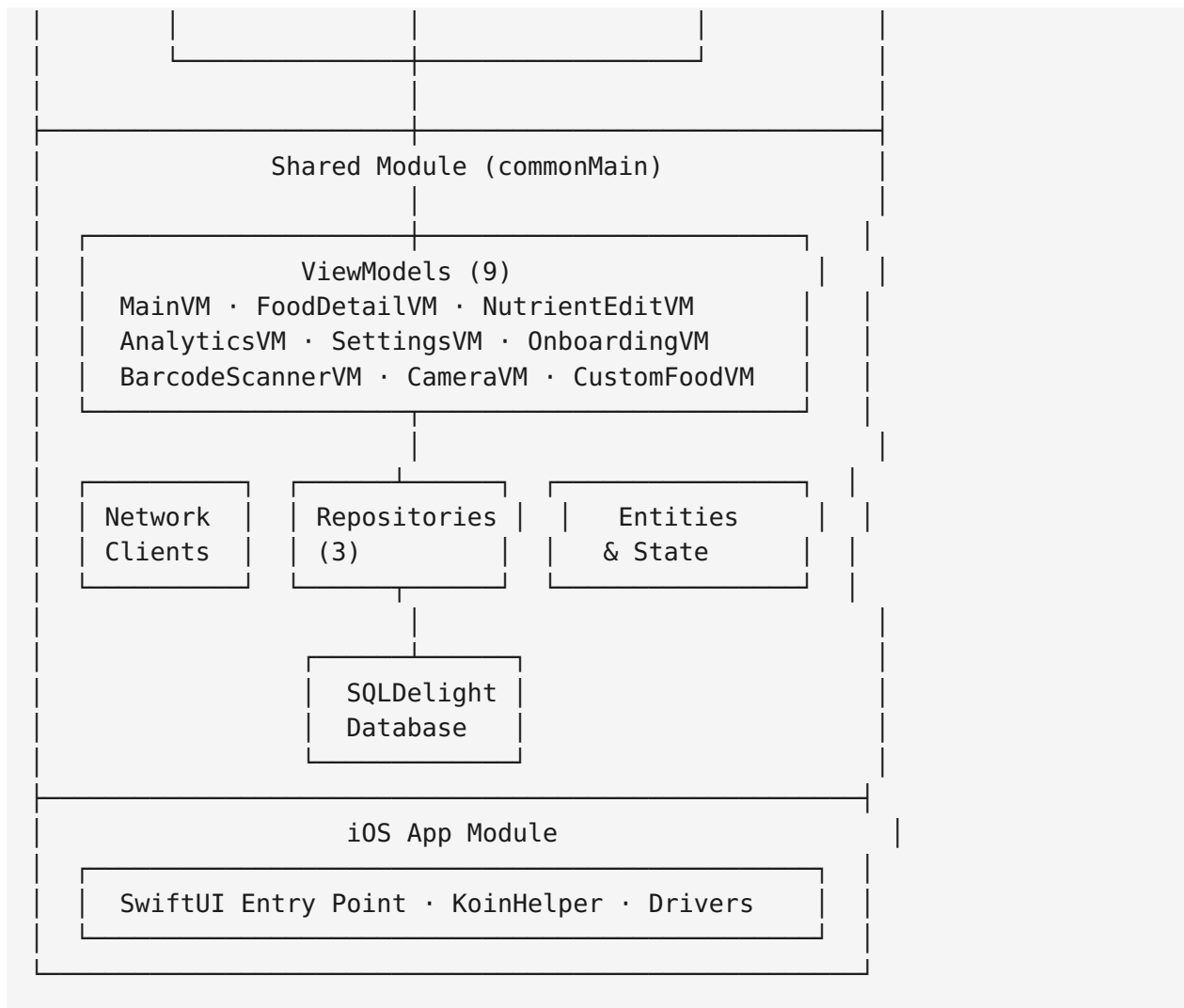
- **JDK:** 21 (JDK 25 is NOT compatible with AGP 8.7.2)
- **Android Gradle Plugin:** 8.7.2
- **Minimum SDK:** API 24 (Android 7.0)
- **Compile/Target SDK:** 35

5. Architecture

5.1 MVVM Pattern

The application follows the Model-View-ViewModel pattern with a clear separation between shared business logic and platform-specific UI:





5.2 DI Module Structure

- **SharedModule** (commonMain): Registers singletons for repositories, network clients, database, and utilities
- **AppModule** (Android): Registers ViewModels (via `viewModel { }`), platform implementations (DriverFactory, AndroidImageStorage)

5.3 Data Model

Three SQLite tables managed by SQLDelight:

food_items — Stores all food entries (AI-analyzed, barcode-scanned, manual, custom):

Column	Type	Description
id	INTEGER PK	Auto-increment identifier
name	TEXT	Food name (default: "Neznámé jídlo")
calories	INTEGER	Total calories

Column	Type	Description
protein	INTEGER	Protein in grams
fat	INTEGER	Fat in grams
carbs	INTEGER	Carbohydrates in grams
portion	INTEGER	Number of portions
healthScore	INTEGER	AI health score (1–10)
createdAt	TEXT	ISO-8601 creation timestamp
updatedAt	TEXT	ISO-8601 update timestamp
localImagePath	TEXT	Path to stored food photo
loading	INTEGER	1 = analysis in progress, 0 = complete
isCustom	INTEGER	1 = user-created, 0 = analyzed/scanned

personal_info — Historical user body metrics (new row per update for trend tracking):

Column	Type	Description
id	INTEGER PK	Auto-increment identifier
gender	TEXT	"male" or "female"
age	INTEGER	User age in years
heightCm	REAL	Height in centimeters
weightKg	REAL	Weight in kilograms
activityLevel	INTEGER	1 = Sedentary, 2 = Light, 3 = Active, 4 = Very Active
createdAt	TEXT	ISO-8601 creation timestamp
updatedAt	TEXT	ISO-8601 update timestamp

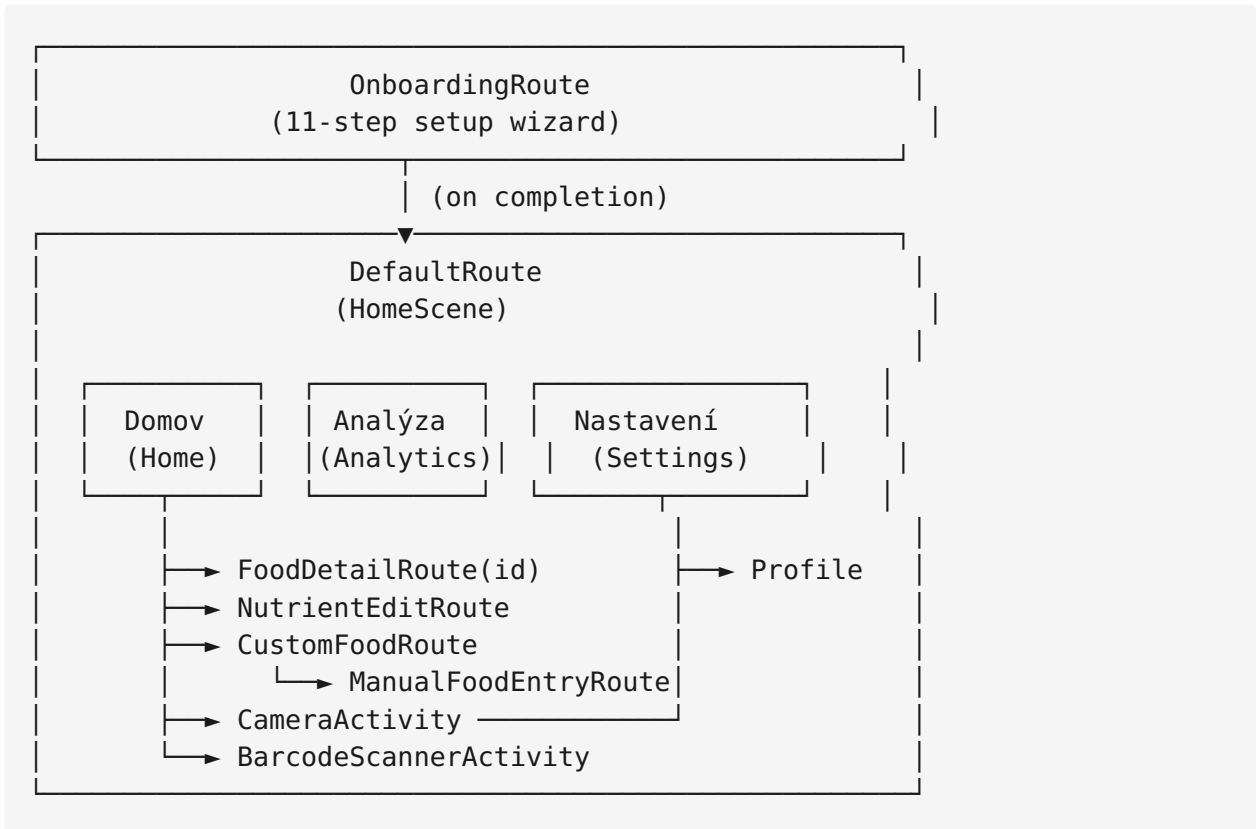
nutrient_settings — Daily nutritional targets:

Column	Type	Description
id	INTEGER PK	Auto-increment identifier
targetCalories	INTEGER	Daily calorie target
targetProtein	INTEGER	Daily protein target (grams)
targetFat	INTEGER	Daily fat target (grams)
targetCarbs	INTEGER	Daily carbohydrate target (grams)

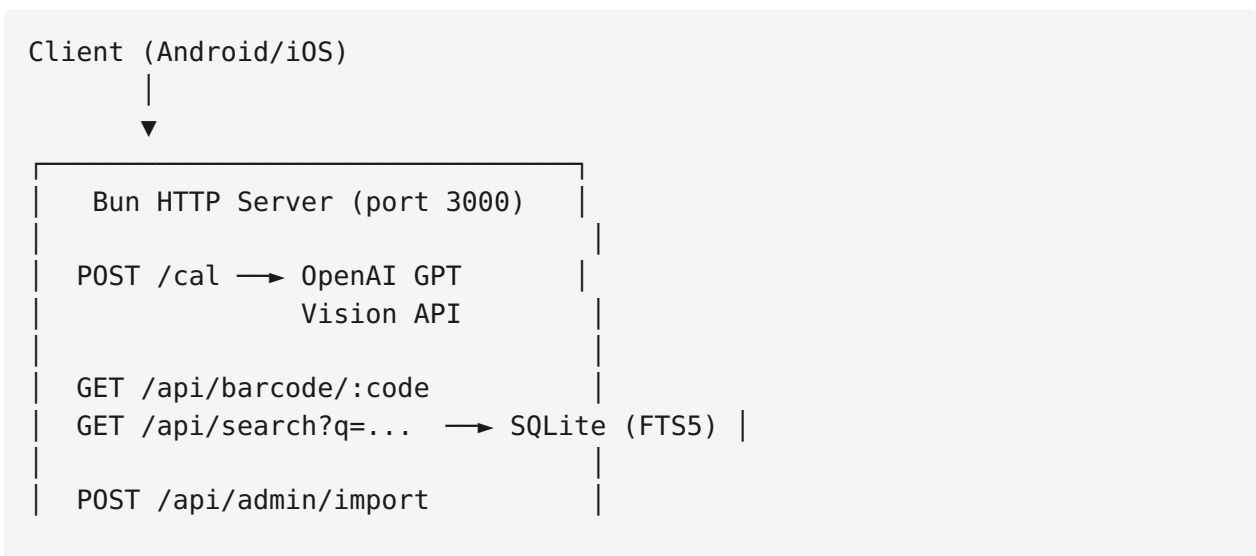
Column	Type	Description
createdAt	TEXT	ISO-8601 creation timestamp
updatedAt	TEXT	ISO-8601 update timestamp

5.4 Navigation Graph

Type-safe navigation using `@Serializable` route objects with Compose Navigation:



5.5 Backend Architecture



6. UI Screen Descriptions

6.1 Onboarding (OnboardingRoute)

An 11-step wizard presented on first launch. Each step fills the full screen with a centered input control. A progress bar at the top shows completion percentage. Users navigate via "Continue" / "Back" buttons or swipe gestures. The final step shows "Done" to complete setup.

6.2 Home Screen (DefaultRoute — Domov tab)

The primary daily tracking dashboard. At the top, a **week date picker** lets users navigate between days with a "Today" shortcut. Below it, a **calorie card** displays current vs. target calories with a progress bar and the current streak count. Three **macronutrient cards** (protein, carbs, fat) show individual progress with circular indicators and custom icons (chicken leg, wheat, avocado). The lower section contains a scrollable list of **recently added food items** for the selected date, each showing the food image, name, calories, and macros. Tapping a food opens the detail view; long-pressing activates **selection mode** with a bottom action bar for bulk save-as-custom or delete. An **add button** navigates to the custom food search screen. Two **floating action buttons** at the bottom provide quick access to camera capture and barcode scanning.

6.3 Food Detail (FoodDetailRoute)

A full-screen food detail view. The top 52% displays the **food photograph** with translucent back and menu buttons overlaid. A **bottom sheet card** slides up containing editable fields: food name (text input), calories (read-only), protein, fat, and carbs (editable numeric inputs). A **"Fix Result"** button re-sends the image for AI re-analysis. A **"Finish"** button saves changes and navigates back. The three-dot menu offers **Share** and **Delete** actions.

6.4 Nutrient Edit (NutrientEditRoute)

A screen for adjusting daily macronutrient targets. A **vertical calorie card** at the top shows the current calorie count and progress ratio. Below, a **macronutrient card** contains three expandable rows for protein, carbs, and fat — each with a label, current value in grams, a color-coded icon, and an iOS-style **wheel picker** (0–500g range) that expands inline when tapped.

6.5 Custom Food Search (CustomFoodRoute)

A food search and selection screen. A **search bar** at the top filters results. A **segmented control** switches between three tabs: All (combined results), My Foods (user-created), and Recently Used. The food list shows items with checkboxes, thumbnail images, names, calories, and macros. In the "My Foods" tab, an **"Add Manually"** button navigates to manual food entry. Online search results from the API include an add button that opens a **portion picker bottom sheet** with quick portion buttons (50g, 100g, 150g, 200g, 250g) and scaled nutrition preview. A floating **add button** at the bottom shows the selection count and adds all selected foods.

6.6 Manual Food Entry (ManualFoodEntryRoute)

A form for creating custom foods from scratch. Includes a **food name input**, an **ingredient search** with auto-suggestions from the database and API (top 5 results), and a **source foods card** showing added ingredients with adjustable portion sizes and auto-calculated macros. A **calories summary** card shows the total. Expandable **macro editors** (protein, carbs, fat) use inline wheel pickers (0–500g). The **save button** is disabled until a food name is provided.

6.7 Analytics (Analýza tab)

A data visualization screen. Two collapsible **date range pickers** (start and end) use iOS-style inline wheel selectors. A **weight line chart** displays weight trends across the selected date range. **Nutrition bar charts** show daily calorie and macronutrient totals.

6.8 Settings (Nastavení tab)

A configuration screen organized in sections: - **Appearance**: Segmented control for System / Light / Dark theme - **Language**: Toggle between Czech and English - **Units**: Toggle between Metric and Imperial - **Notifications**: Switch to enable/disable meal reminders (triggers Android 13+ permission request) - **Account**: Displays app version

6.9 Profile (accessible from Settings)

User body metrics editor. A **personal info card** with three rows (weight, height, age) using inline pickers. A **gender selector** (segmented control). A **BMI indicator card** showing the calculated BMI value and category. An **activity level card** with four selectable rows and checkmark indicators.

6.10 Camera (CameraActivity)

A full-screen camera interface. The **CameraX preview** fills the screen with a translucent **back button** at the top. A **mode toggle** at the bottom switches between PHOTO and QR modes. A large **capture button** triggers photo capture. In QR mode, detected barcodes

trigger a product lookup with a **result overlay** showing loading state, product information with portion input, or error state with retry option.

6.11 Barcode Scanner (BarcodeScannerActivity)

A dedicated barcode scanning interface. The **camera preview** fills the background with a **close button** at the top-left. A hint text ("Naskenuj čárový kód") displays during scanning. A **bottom card overlay** shows scan results: loading spinner during lookup, product details with quantity input and macros when found, or a not-found/error message with retry button.

7. API Specification

7.1 Food Analysis

```
POST /cal
Content-Type: image/*
Body: Raw image bytes

Response: {
  "weight": 250,
  "foodType": "main_dish",
  "title": "Kuřecí řízek s bramborovou kaší",
  "protein": 35,
  "fat": 18,
  "carbs": 42,
  "healthScore": 7
}
```

7.2 Barcode Lookup

```
GET /api/barcode/:code

Response (200): {
  "id": 1,
  "barcode": "8593893749320",
  "name": "Jogurt bílý",
  "energy_kcal_100g": 57,
  "protein_100g": 10,
  "fat_100g": 0.8,
  "carbs_100g": 4,
  "serving_size": "150g",
  "image_url": "...
}
```

```
Response (404): { "error": "Product not found" }
```

7.3 Product Search

```
GET /api/search?q=jogurt
```

```
Response: [  
  { "barcode": "...", "name": "Jogurt bílý", ... },  
  ...  
] (max 20 results)
```

7.4 Open Food Facts Fallback

```
GET https://world.openfoodfacts.org/api/v2/product/{barcode}.json
```

Used as fallback when the local backend database returns no results for a barcode.